Unit 2: Functions, Modules and Files

Practical 1: Library Management using Functions

Description:
Develop a menu-driven library management system using user-defined functions for issuing books, returning books, searching books, and displaying available books.

Concepts Covered

- Functions
- Arguments
- Return Values
- Modular Programming

INPUT CODE:

```
print("ENROLLMENT NO. : 250283152004")

books = {
    "B101": {"title": "Python Basics", "copies": 5},
    "B102": {"title": "Data Structures", "copies": 3},
    "B103": {"title": "DBMS Concepts", "copies": 4},
    "B104": {"title": "Operating Systems", "copies": 2}
}

issued_books = {}

def display_books():
    print("\n" + "-" * 50)
    print("{:<10}{:<25}{:<10}".format("ID", "Title", "Copies"))
    print("-" * 50)
    for bid, info in books.items():
        print("{:<10}{:<25}{:<10}".format(bid, info["title"], info["copies"]))
    print("-" * 50 + "\n")

def issue_book(book_id, student_name):
    if book_id not in books:
        print("Book ID not found!\n")
        return
    if books[book_id]["copies"] <= 0:
        print("No copies available for this book!\n")
        return
    books[book_id]["copies"] -= 1
    issued_books[book_id] = issued_books.get(book_id, [])
    issued_books[book_id].append(student_name)
    print("Book issued to " + student_name + " successfully!\n")

def return_book(book_id, student_name):
    if book_id not in issued_books or student_name not in issued_books[book_id]:
        print("No record found for this issue!\n")
        return
```

```
    issued_books[book_id].remove(student_name)
    books[book_id]["copies"] += 1
    print("Book returned successfully!\n")

def search_book(book_id):
    if book_id in books:
        info = books[book_id]
        print("\nTitle: " + info["title"] + " | Copies Available: " + str(info["copies"]
    else:
        print("Book not found!\n")

while True:
    print("===== LIBRARY MANAGEMENT SYSTEM =====")
    print("1. Display All Books")
    print("2. Issue Book")
    print("3. Return Book")
    print("4. Search Book")
    print("5. Exit")

    choice = input("Enter your choice (1-5): ")

    if choice == "1":
        display_books()
    elif choice == "2":
        bid = input("Enter Book ID: ")
        sname = input("Enter Student Name: ")
        issue_book(bid, sname)
    elif choice == "3":
        bid = input("Enter Book ID: ")
        sname = input("Enter Student Name: ")
        return_book(bid, sname)
    elif choice == "4":
        bid = input("Enter Book ID to search: ")
        search_book(bid)
    elif choice == "5":
        print("Exiting... Goodbye!")
        break
    else:
        print("Invalid choice! Try again.\n")
```

OUTPUT CODE:

```
ENROLLMENT NO. : 250283152004
===== LIBRARY MANAGEMENT SYSTEM =====
1. Display All Books
2. Issue Book
3. Return Book
4. Search Book
5. Exit
Enter your choice (1-5): 1

--------------------------------------------------
ID        Title                   Copies
--------------------------------------------------
B101      Python Basics           5
B102      Data Structures         3
B103      DBMS Concepts           4
B104      Operating Systems       2
--------------------------------------------------

===== LIBRARY MANAGEMENT SYSTEM =====
1. Display All Books
2. Issue Book
3. Return Book
4. Search Book
5. Exit
Enter your choice (1-5):
```

Practical 2: Payroll Processing System

Description:

Create a payroll system where salary, HRA, DA, PF, tax deduction, and net salary are calculated using separate functions.

Concepts Covered

- Functions
- Default Arguments
- Multiple Return Values

INPUT CODE:

```
print("ENROLLMENT NO. : 250283152004")

def calculate_hra(basic, hra_percent=40):
    return basic * hra_percent / 100

def calculate_da(basic, da_percent=10):
    return basic * da_percent / 100

def calculate_pf(basic, pf_percent=12):
    return basic * pf_percent / 100

def calculate_tax(gross_salary):
    if gross_salary <= 25000:
        return 0
    elif gross_salary <= 50000:
        return gross_salary * 0.05
    else:
        return gross_salary * 0.10

def calculate_net_salary(basic):
    hra = calculate_hra(basic)
    da = calculate_da(basic)
    pf = calculate_pf(basic)
    gross_salary = basic + hra + da
    tax = calculate_tax(gross_salary)
    net_salary = gross_salary - pf - tax
    return hra, da, pf, tax, gross_salary, net_salary

print("----- PAYROLL PROCESSING SYSTEM -----")
emp_name = input("Enter Employee Name: ")
basic_salary = float(input("Enter Basic Salary: "))

hra, da, pf, tax, gross_salary, net_salary = calculate_net_salary(basic_salary)

print("\n" + "=" * 40)
print("            PAYSLIP")
print("=" * 40)
print("{:<20}: {}".format("Employee Name", emp_name))
print("{:<20}: {:.2f}".format("Basic Salary", basic_salary))
```

```
print("\n" + "=" * 40)
print("            PAYSLIP")
print("=" * 40)
print("{:<20}: {}".format("Employee Name", emp_name))
print("{:<20}: {:.2f}".format("Basic Salary", basic_salary))
print("{:<20}: {:.2f}".format("HRA", hra))
print("{:<20}: {:.2f}".format("DA", da))
print("{:<20}: {:.2f}".format("Gross Salary", gross_salary))
print("{:<20}: {:.2f}".format("PF Deduction", pf))
print("{:<20}: {:.2f}".format("Tax Deduction", tax))
print("-" * 40)
print("{:<20}: {:.2f}".format("Net Salary", net_salary))
print("=" * 40)
```

OUTPUT CODE:

```
ENROLLMENT NO. : 250283152004
----- PAYROLL PROCESSING SYSTEM -----
Enter Employee Name: jay
Enter Basic Salary: 5000

=======================================
             PAYSLIP
=======================================
Employee Name       : jay
Basic Salary        : 5000.00
HRA                 : 2000.00
DA                  : 500.00
Gross Salary        : 7500.00
PF Deduction        : 600.00
Tax Deduction       : 0.00
---------------------------------------
Net Salary          : 6900.00
=======================================
```

Practical 3: Student Result Analysis using Lambda Functions

Description:
Store student marks and use lambda functions with map(), filter(), and reduce() to calculate grades, identify toppers, and compute class averages.

Concepts Covered

- Lambda Functions
- map()
- filter()
- reduce()

INPUT CODE:

```
print("ENROLLMENT NO. : 250283152004")

from functools import reduce

students = {
    "Amit": 78,
    "Priya": 92,
    "Rahul": 45,
    "Sneha": 67,
    "Vikas": 88
}

names = list(students.keys())
marks = list(students.values())

grades = list(map(lambda m: "A+" if m >= 90 else "A" if m >= 80 else "B" if m >= 70 else "C" if m >= 60 else "D" if m >= 50 else "Fail", marks))

toppers = list(filter(lambda item: item[1] >= 80, students.items()))

total_marks = reduce(lambda x, y: x + y, marks)
class_average = total_marks / len(marks)

print("----- STUDENT RESULT ANALYSIS -----\n")

print("{:<10}{:<10}{:<10}".format("Name", "Marks", "Grade"))
print("-" * 30)
for i in range(len(names)):
    print("{:<10}{:<10}{:<10}".format(names[i], marks[i], grades[i]))

print("\n----- TOPPERS (Marks >= 80) -----")
for name, mark in toppers:
    print(name + ": " + str(mark))

print("\nClass Average: {:.2f}".format(class_average))
print("Highest Marks: " + str(max(marks)) + " (" + names[marks.index(max(marks))] + ")")
print("Lowest Marks: " + str(min(marks)) + " (" + names[marks.index(min(marks))] + ")")
```

OUTPUT CODE:

```
ENROLLMENT NO. : 250283152004
----- STUDENT RESULT ANALYSIS -----

Name      Marks     Grade
------------------------------
Amit      78        B
Priya     92        A+
Rahul     45        Fail
Sneha     67        C
Vikas     88        A

----- TOPPERS (Marks >= 80) -----
Priya: 92
Vikas: 88

Class Average: 74.00
Highest Marks: 92 (Priya)
Lowest Marks: 45 (Rahul)
```

Practical 4: Employee Attendance Module

Description:
Create your own Python module to calculate employee attendance percentage, working days, leaves, and overtime.

Concepts Covered

- Modules
- Import Statement
- Functions

INPUT CODE:

```python
# attendance_module.py

def working_days(total_days, weekly_offs):
    return total_days - weekly_offs

def attendance_percentage(days_present, total_working_days):
    return (days_present / total_working_days) * 100

def leaves_taken(total_working_days, days_present):
    return total_working_days - days_present

def calculate_overtime(hours_worked, standard_hours=8):
    if hours_worked > standard_hours:
        return hours_worked - standard_hours
    return 0
```

```python
# main.py
print("ENROLLMENT NO. : 250183152004")

print("----- EMPLOYEE ATTENDANCE MODULE -----")

emp_name = input("Enter Employee Name: ")
total_days = int(input("Enter Total Days in Month: "))
weekly_offs = int(input("Enter Number of Weekly Offs: "))
days_present = int(input("Enter Days Present: "))
hours_worked = float(input("Enter Average Hours Worked per Day: "))

work_days = working_days(total_days, weekly_offs)
percentage = attendance_percentage(days_present, work_days)
leaves = leaves_taken(work_days, days_present)
overtime = calculate_overtime(hours_worked)

print("\n" + "=" * 40)
print("        ATTENDANCE REPORT")
print("=" * 40)
print("{:<20}: {}".format("Employee Name", emp_name))
print("{:<20}: {}".format("Total Working Days", work_days))
print("{:<20}: {}".format("Days Present", days_present))
print("{:<20}: {}".format("Leaves Taken", leaves))
print("{:<20}: {:.2f}%".format("Attendance Percentage", percentage))
print("{:<20}: {} hrs/day".format("Overtime", overtime))
print("=" * 40)
```

X

OUTPUT CODE:

```
ENROLLMENT NO. : 250283152004
----- EMPLOYEE ATTENDANCE MODULE -----
Enter Employee Name: jY
Enter Total Days in Month: 27
Enter Number of Weekly Offs: 1
Enter Days Present: 9
Enter Average Hours Worked per Day: 7

========================================
        ATTENDANCE REPORT
========================================
Employee Name        : jY
Total Working Days   : 26
Days Present         : 9
Leaves Taken         : 17
Attendance Percentage: 34.62%
Overtime             : 0.0 hrs/day
========================================
```

Practical 5: Electricity Consumption Report Generator

Description:
Develop a program that reads monthly electricity consumption from a file, processes the data using functions, and generates a summary report.

Concepts Covered

- Functions
- Files
- Modules

INPUT CODE:

```
# consumption.txt

January,250
February,180
March,320
April,410
May,150
```

```python
# report_module.py
def read_consumption_data(filename):
    data = {}
    with open(filename, "r") as file:
        for line in file:
            line = line.strip()
            if line:
                month, units = line.split(",")
                data[month] = float(units)
    return data

def calculate_bill(units):
    if units <= 100:
        return units * 3.50
    elif units <= 300:
        return (100 * 3.50) + (units - 100) * 5.00
    else:
        return (100 * 3.50) + (200 * 5.00) + (units - 300) * 7.50

def generate_report(data):
    report_lines = []
    total_units = 0
    total_bill = 0
    for month, units in data.items():
        bill = calculate_bill(units)
        total_units += units
        total_bill += bill
        report_lines.append("{:<12}{:<12}Rs.{:<10.2f}".format(month, units, bill))
    return report_lines, total_units, total_bill
```

```python
print("ENROLLMENT NO. : 250183152004")

# The functions were defined in a previous cell and are globally available.
# import report_module as rm

filename = "consumption.txt"
data = read_consumption_data(filename)
report_lines, total_units, total_bill = generate_report(data)

print("\n" + "=" * 45)
print("     ELECTRICITY CONSUMPTION REPORT")
print("=" * 45)
print("{:<12}{:<12}{:<12}".format("Month", "Units", "Bill"))
print("-" * 45)
for line in report_lines:
    print(line)
print("-" * 45)
print("Total Units Consumed: " + str(total_units))
print("Total Bill Amount: Rs.{:.2f}".format(total_bill))
print("Average Monthly Bill: Rs.{:.2f}".format(total_bill / len(data)))
print("=" * 45)

with open("summary_report.txt", "w") as f:
    f.write("ELECTRICITY CONSUMPTION SUMMARY REPORT\n")
    f.write("=" * 45 + "\n")
    for line in report_lines:
        f.write(line + "\n")
    f.write("-" * 45 + "\n")
    f.write("Total Units Consumed: " + str(total_units) + "\n")
    f.write("Total Bill Amount: Rs." + "{:.2f}".format(total_bill) + "\n")

print("\nSummary report saved to summary_report.txt")
```

```
ENROLLMENT NO. : 250283152004

============================================
     ELECTRICITY CONSUMPTION REPORT
============================================
Month       Units       Bill
--------------------------------------------
January     250.0       Rs.1100.00
February    180.0       Rs.750.00
March       320.0       Rs.1500.00
April       410.0       Rs.2175.00
May         150.0       Rs.600.00
--------------------------------------------
Total Units Consumed: 1310.0
Total Bill Amount: Rs.6125.00
Average Monthly Bill: Rs.1225.00
============================================

Summary report saved to summary_report.txt
```

Practical 6: Bank Interest Calculator using Recursive Functions

Description:
Implement recursive functions to calculate compound interest over multiple years and compare it with simple interest.

Concepts Covered

- Recursion
- Functions

INPUT CODE:

```
print("ENROLLMENT NO. : 250183152004")

def simple_interest(principal, rate, time):
    return (principal * rate * time) / 100

def compound_interest_recursive(principal, rate, years):
    if years == 0:
        return principal
    amount_so_far = compound_interest_recursive(principal, rate, years - 1)
    return amount_so_far + (amount_so_far * rate / 100)

print("----- BANK INTEREST CALCULATOR -----")

principal = float(input("Enter Principal Amount: "))
rate = float(input("Enter Rate of Interest (%): "))
years = int(input("Enter Number of Years: "))

si = simple_interest(principal, rate, years)
final_amount_ci = compound_interest_recursive(principal, rate, years)
ci = final_amount_ci - principal

print("\n" + "=" * 45)
print("        INTEREST COMPARISON REPORT")
print("=" * 45)
print("{:<25}: Rs.{:.2f}".format("Principal Amount", principal))
print("{:<25}: {}%".format("Rate of Interest", rate))
print("{:<25}: {} years".format("Time Period", years))
print("-" * 45)
print("{:<25}: Rs.{:.2f}".format("Simple Interest", si))
print("{:<25}: Rs.{:.2f}".format("Amount (SI)", principal + si))
print("-" * 45)
print("{:<25}: Rs.{:.2f}".format("Compound Interest", ci))
print("{:<25}: Rs.{:.2f}".format("Amount (CI)", final_amount_ci))
print("-" * 45)
```

```
if ci > si:
    print("Compound Interest yields Rs.{:.2f} more than Simple Interest".format(ci - si))
else:
    print("Simple Interest yields Rs.{:.2f} more than Compound Interest".format(si - ci))
print("=" * 45)
```

OUTPUT CODE:

```
ENROLLMENT NO. : 250283152004
----- BANK INTEREST CALCULATOR -----
Enter Principal Amount: 2000
Enter Rate of Interest (%): 6
Enter Number of Years: 4

==========================================
        INTEREST COMPARISON REPORT
==========================================
Principal Amount         : Rs.2000.00
Rate of Interest         : 6.0%
Time Period              : 4 years
------------------------------------------
Simple Interest          : Rs.480.00
Amount (SI)              : Rs.2480.00
------------------------------------------
Compound Interest        : Rs.524.95
Amount (CI)              : Rs.2524.95
------------------------------------------
Compound Interest yields Rs.44.95 more than Simple Interest
==========================================
```